

DATE : 02.10.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: SHELL SCRIPTING
CONDITIONAL STATEMENTS
SESSION : 9
BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Table of Contents



- ▶ How if works in Bash
- ▶ Numeric Comparisons
- ▶ String Comparisons
- ▶ File Tests
- ▶ Logical Operators
- ▶ Command Status Conditions
- ▶ if / else
- ▶ if / elif / else
- ▶ Nested if
- ▶ read + if
- ▶ Positional Parameters + if
- ▶ exit & Exit Status
- ▶ Practical Exercises

From Day 8 to Day 9

Day 8: variables, input, parameters, arithmetic, and exit status.

Day 9: use those values to make decisions.

Key idea:

Bash if evaluates the exit status of a command or test.

0 → success / condition true

non-zero → failure / condition false



If Statements



Use if when you want commands to run only when a condition succeeds.

Bash checks the condition's exit status.

0 → true/success

non-zero → false/failure

Preferred Bash test syntax:

```
[[ condition ]]
```

```
if [[ condition ]]; then  
    commands
```

```
fi
```

Example

```
if [[ -f "$FILE" ]]; then  
    echo "File exists"
```

```
fi
```

Numeric Comparison Operators

- eq equal
- ne not equal
- lt less than
- le less than or equal
- gt greater than
- ge greater than or equal

Use these for numeric comparisons.

```
CPU=75
```

```
if [[ $CPU -gt 80 ]]; then  
    echo "High CPU usage"  
else  
    echo "CPU usage is normal"  
fi
```

String Comparison Operators

`==` equal

`!=` not equal

`-z` string is empty

`-n` string is not empty

Use quotes around variables when comparing strings.

```
ENVIRONMENT="production"
```

```
if [[ "$ENVIRONMENT" == "production" ]];  
then
```

```
    echo "Production environment"
```

```
fi
```

```
if [[ -z "$USERNAME" ]]; then  
    echo "Username is empty"
```

```
fi
```

File Test Operators

- e exists
- f regular file
- d directory
- r readable
- w writable
- x executable
- s exists and is not empty

These tests connect directly to the permissions and file-management work from earlier days.

```
FILE="deploy.sh"

if [[ -f "$FILE" ]]; then
    echo "Regular file"
fi

if [[ -x "$FILE" ]]; then
    echo "Executable"
fi
```

Logical Operators

&& AND — both conditions must be true

|| OR — at least one condition must be true

! NOT — reverses a condition

Use logical operators to combine tests.

```
if [[ $CPU -gt 80 && $MEMORY -gt 80 ]]; then
    echo "Resources are high"
fi
```

```
if [[ "$ENV" == "production" || "$ENV" ==
    "staging" ]]; then
    echo "Deployment environment"
fi
```

```
if [[ ! -f "$FILE" ]]; then
    echo "File missing"
fi
```

Command Status Conditions

An if statement can test a command directly.

The command succeeds → the then block runs.

The command fails → the else block can run.

This is common in server checks and automation.

```
if systemctl is-active --quiet sshd; then
    echo "SSH service is running"
else
    echo "SSH service is not running"
fi
```

```
if command -v curl > /dev/null 2>&1; then
    echo "curl is installed"
fi
```

If-Else Statements

Use if-else when there are two possible paths.

One block runs when the condition succeeds.

The other block runs when it fails.

Use `[[ ]]` consistently in this course.

```
FILE="deployment.log"

if [[ -f "$FILE" ]]; then
    echo "$FILE exists."
else
    echo "$FILE does not exist."
    touch "$FILE"
fi
```

If-Elif-Else Statements

Use elif when you need to test multiple conditions.

Bash checks the conditions from top to bottom.

The first matching condition runs.

If none match, else runs.

```
ENVIRONMENT="production"

if [[ "$ENVIRONMENT" == "development" ]]; then
    echo "Development"
elif [[ "$ENVIRONMENT" == "staging" ]]; then
    echo "Staging"
elif [[ "$ENVIRONMENT" == "production" ]]; then
    echo "Production"
else
    echo "Unknown environment"
fi
```

Nested If Statements

A nested if places one conditional inside another.

Use it when a second decision only matters after the first condition succeeds.

Example: first check whether a deployment script exists; then check whether it is executable.

```
FILE="deploy.sh"

if [[ -e "$FILE" ]]; then
    if [[ -x "$FILE" ]]; then
        echo "Exists and executable"
    else
        echo "Exists but not executable"
    fi
else
    echo "Script does not exist"
fi
```

read + if

Reuse the input skills from Day 8.

1. Read a value.
2. Compare it.
3. Choose an action.

DevOps example: warn when CPU usage is above a threshold.

```
read -r -p "Enter CPU usage: " CPU

if [[ $CPU -gt 80 ]]; then
    echo "WARNING: High CPU usage"
else
    echo "CPU usage is normal"
fi
```

Positional Parameters + if

Combine Day 8 parameters with Day 9 conditions.

Use \$# to validate the number of arguments.
Use \$1 to inspect the requested environment.
Use exit 1 when the usage is invalid.

```
#!/usr/bin/env bash

if [[ $# -ne 1 ]]; then
    echo "Usage: $0 <environment>"
    exit 1
fi

if [[ "$1" == "production" ]]; then
    echo "Production deployment"
else
    echo "Non-production deployment"
fi
```

exit & Exit Status

Connect today's conditions to Day 8 exit-status concepts.

exit 0 → successful completion

exit non-zero → script reports failure

This is important for automation and CI/CD pipelines.

```
if [[ $# -ne 1 ]]; then
    echo "Usage: $0 <environment>"
    exit 1
fi

echo "Argument accepted"
exit 0
```

Which Test Should I Use?

Number? → -eq -ne -lt -le -gt -ge

String? → == != -z -n

File? → -e -f -d -r -w -x -s

Command? → if command; then ...

Traditional [] / test syntax exists, but this course uses [[]] as the main Bash syntax.

```
if [[ $COUNT -gt 5 ]]; then ...
```

```
if [[ "$ENV" == "production" ]]; then ...
```

```
if [[ -f "$FILE" ]]; then ...
```

```
if command -v curl >/dev/null 2>&1; then  
...
```

Exercise 1 — File Check

Create file-check.sh.

Ask the user for a filename.

Requirements:

1. If it does not exist, report that it does not exist.
2. If it is a regular file, report "Regular file".
3. If it is a directory, report "Directory".
4. If it is executable, report that too.

Use read -r -p and file tests.

```
#!/usr/bin/env bash
```

```
read -r -p "Enter path: " PATHNAME
```

```
# Build the conditions yourself.
```

Exercise 2 — Server Health Check

Create health-check.sh.

Check:

1. The root directory exists.
2. The SSH service is active.
3. curl is installed.
4. A selected log file exists.

Print a clear PASS/FAIL message for each check.

Use command-status conditions and file tests.

```
if systemctl is-active --quiet sshd; then
    echo "PASS: SSH"
else
    echo "FAIL: SSH"
fi

# Add the remaining checks.
```

Exercise 3 — Environment Validator

Run:

`./environment-check.sh production`

The script must recognize:

development

staging

production

Anything else must be rejected.

Use `$1`, `$#`, `if`, `elif`, `else`, and `exit`.

```
if [[ $# -ne 1 ]]; then
    echo "Usage: $0 <environment>"
    exit 1
fi

# Add development/staging/production checks.
```

Exercise 4 — Deployment Safety Check

Create deploy-check.sh.

Before deployment:

1. Check that the deployment script exists.
2. Check that it is executable.
3. Check that the configuration file exists.
4. Check that the environment is valid.

If everything passes, print:

All deployment checks passed.

Ready to deploy.

Otherwise print:

Deployment checks failed.

Deployment aborted.

```
# Combine file tests, string tests,  
# command/argument checks, and  
# logical operators.
```

```
# Do not deploy if any check fails.
```

Day 9 Review

if → one decision

if / else → two paths

if / elif / else → multiple paths

nested if → decision inside a decision

Numbers → -eq -ne -lt -le -gt -ge

Strings → == != -z -n

Files → -e -f -d -r -w -x -s

Commands → test command exit status

Combine tests with &&, ||, !

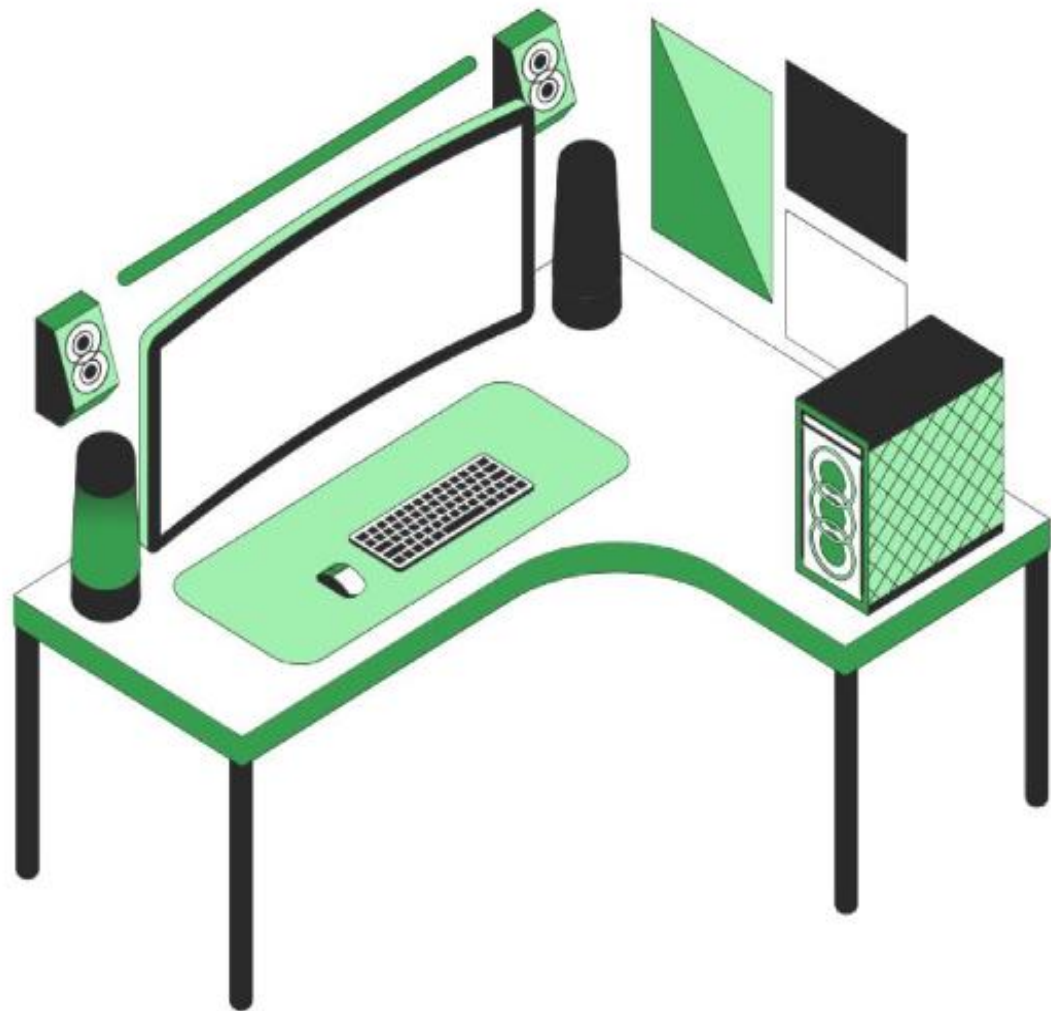
Use exit 0 / non-zero for script results.

```
if [[ condition ]]; then ...
```

```
elif [[ condition ]]; then ...
```

```
else ...
```

```
fi
```



Do you have any questions?

Send it to us! We hope you learned something new.